Malware Analysis

Malware Analysis Demo — Static & Dynamic Analysis

Satan Ransomware (PE32) on REMnux

Prepared By Anirudh Revanur

Overview

This writeup documents a live malware analysis demo performed on REMnux using a Satan ransomware sample. It covers the full workflow from static analysis through dynamic detonation, post-detonation triage, and a cross-environment comparison using any.run sandbox reports.

The core lesson: Static analysis tells you what malware *can* do. Dynamic analysis shows you what it *actually does.*

Sample Information

Field	Value
Filename	`28f299818268701eb239974b74b3d80fc28e2d14cc0f45a52a9bc62be784d170.exe.bin`
Family	Satan Ransomware
Type	PE32 executable (GUI) Intel 80386
Compiled	2017-02-23 19:28:24 UTC
MD5	`f592e7faba96a23ee25ff25f3779f44f`
SHA1	`66656b81b43130ddd6774845c249700ffb06cead`
SHA256	`28f299818268701eb239974b74b3d80fc28e2d14cc0f45a52a9bc62be784d170`
SSDEEP	`3072:HeIBtQnE7OhssdWJ5jy392aCmCbBq+RdVzp9xMwjUvZT3+fQRJZHjhyz:vqvhssdu5jyYa`

Environment

- **Analysis OS:** REMnux (Ubuntu-based Linux malware analysis distro)

- **Execution layer:** Wine (Windows compatibility layer)
- **Network interface:** `enp0s3` (10.0.2.15/24)
- **Sandbox reference:** any.run (Windows 7 SP1 x32 and Windows 10 Pro x64)

> **Golden rule:** Always snapshot your VM *before* detonation. Restore to snapshot after each run.

Tools Used

Tool	Purpose
`peframe`	PE file analysis — hashes, entropy, imports, anti-debug
`pehash`	Hash generation (MD5, SHA1, SHA256, SSDEEP)
`pefile` (Python)	Programmatic PE inspection
`inotifywait`	Real-time filesystem event monitoring
`tcpdump`	Raw network packet capture
`tshark`	Pcap analysis and filtering
`wine`	Running Windows PE binaries on Linux

Act 1 — Static Analysis

Static analysis means examining the file without executing it. Zero risk, runs on your host.

File identification

When we wanna identify a file, the first thing we do is have a look at the extensions. The problem with this extension is that it's a part of the file name, and can be changed.

In order to make sure that we can identify the file type without any error is through a sequence of Magic Bytes

Magic Bytes

Magic Bytes are the first few bytes that create the header and tell the operating system what kind of file it actually is

```
mv malware.exe invoice.pdf
```

Now the file that was identified as an executable has now been renamed to become a PDF. The problem with that is that now we believe it's a PDF, but the Operating System knows that it's an executable and treats it like one.

Humans believe the file extension, while the Operating System and the Analysis Tools we use have a look at these Magic Bytes.

```
xxd <file_name> | head
```

This will show us the first ten lines in hexadecimal format and the ASCII values of that. The first few bytes that we see are called as the Magic Bytes or the File Signature.

If you want to have a deeper dive into File Format Magic Bytes, then follow this Guide on GitHub

Let's get on with analyzing our malware sample now

```
file satan.exe
sha256sum satan.exe
md5sum satan.exe
```

PE structure analysis

```
peframe satan.exe
pehash satan.exe
```

Key peframe findings:

Field	Value	Significance
File type	PE32 GUI Intel 80386	32-bit Windows executable
Compile date	2017-02-23 19:28:24	Attribution / age
Packed	Yes (IsPacked Yara)	Content is compressed/obfuscated
`.text` entropy	6.21	High — consistent with packing
`.rdata` entropy	6.08	High — consistent with packing
Features	`antidbg` , `crypto`	Actively resists analysis
Anti-debug APIs	`IsDebuggerPresent` , `FindWindowW` , `Process32FirstW/NextW`	Checks for analysis tools
Crypto APIs	`Advapi Hash API` , `CRC32`	Encryption machinery present

Import analysis

```
python3 check_imports.py
```

Total imports: only 43 — suspiciously low for ransomware. This means most API calls are resolved dynamically at runtime via `GetProcAddress` and `LoadLibraryW`, which are both present. Static import analysis only shows the tip of the iceberg.

Suspicious import clusters to highlight:

Import	What it signals
`VirtualAlloc` + `OpenProcess` + `GetThreadContext`	Process injection primitives
`CreateProcessW` + `GetProcAddress` + `LoadLibraryW`	Dynamic API resolution
`IsDebuggerPresent` + `FindWindowW`	Active anti-analysis checks
`CreateToolhelp32Snapshot` + `Process32FirstW/NextW`	Process enumeration
`DeleteFileW`	File deletion (ransom note cleanup / shadow copies)

String extraction

```
floss satan.exe 2>/dev/null | tee floss_output.txt
grep -iE "zoumot|kased|ukist|zihy|onion|vssadmin|AppData" floss_output.txt
```

> **Note:** FLOSS may return limited results due to runtime string decoding. This is intentional — Satan decodes strings dynamically, which is exactly why dynamic analysis is necessary.

Act 2 — Pre-detonation Setup

Take your VM snapshot NOW before running anything.

Open 4 terminals or terminal tabs. Each one will be performing a different task

Pane 1 — Filesystem watcher

Monitors the Wine fake Windows drive for file drops and directory creation:

```
inotifywait -m -r ~/.wine \
  --format '%T %e %w%f' --timefmt '%H:%M:%S' \
  -e create,modify,delete,move 2>/dev/null
```

Pane 2 — BAT file catcher

Satan drops and deletes a `.bat` file within ~1 second. This catches it mid-flight:

```
inotifywait -m ~/.wine/drive_c/users/remnux/AppData/Local/Temp \
  --format '%w%f' -e create 2>/dev/null | \
  grep "\.bat" | \
  while read f; do cp "$f" ~/captured_$(date +%s).bat 2>/dev/null; done
```

Pane 3 — Network capture

```
sudo tcpdump -i any -n -w satan_capture.pcap 'not port 22'
```

> **Note:** Use `-i any` not `-i enp0s3`. The `any` pseudo-interface avoids BPF filter conflicts with the `LINUX_SLL2` link type.

Pane 4 — Detonation

```
wine ~/28f299818268701eb239974b74b3d80fc28e2d14cc0f45a52a9bc62be784d170.exe.bin
```

Act 3 — Detonation

Fire the sample in Pane 4. Watch Panes 1–3 light up simultaneously.

Immediately after ~60 seconds, kill the network:

```
sudo ip link set enp0s3 down
```

What inotifywait shows (live run results)

```
05:17:51  CREATE,ISDIR  AppData/Roaming/Orcy/            ← persistence directory created
05:17:51  CREATE        AppData/Roaming/Orcy/raru.exe    ← copy of itself dropped
05:17:51  MODIFY x3     AppData/Roaming/Orcy/raru.exe    ← being written to disk
05:17:51  CREATE        AppData/Local/Temp/tmp_31ba24ff.bat  ← BAT file dropped
05:17:52  DELETE        AppData/Local/Temp/tmp_31ba24ff.bat  ← self-deleted after ~1 second
```

```
05:18:04  CREATE       INetCache/.../AKB0EN36          ← C2 response cached
05:18:20  MODIFY x300+  regf710000.tmp                 ← massive registry write (persistence)
```

Key observations:

- The entire drop-and-execute sequence happens in under 2 seconds
- The BAT file self-deletes to cover its tracks
- The INetCache write at `05:18:04` confirms a network response was received
- The registry flood at `05:18:20` is persistence being written

Randomised naming — important teaching point

Satan randomises its persistence directory and filename on every run to evade hash-based detection of dropped files:

Environment	Directory	Dropped EXE
any.run Win7	`Roaming\Zoumot\`	`ukist.exe`
any.run Win10	`Roaming\Kased\`	`zihy.exe`
REMnux (Wine)	`Roaming\Orcy\`	`raru.exe`

Same binary. Same hash. Different persistence path every time.

Act 4 — Post-detonation Triage

Read the captured BAT file

```
cat ~/captured_*.bat
```

This almost certainly contains shadow copy deletion commands:

```
vssadmin delete shadows /all /quiet
bcdedit /set {default} recoveryenabled No
```

> Shadow copy deletion = no free recovery = victim must pay. This is the single most destructive thing ransomware does beyond encryption.

Examine the dropped executable

```
file ~/.wine/drive_c/users/remnux/AppData/Roaming/*/raru.exe
sha256sum ~/.wine/drive_c/users/remnux/AppData/Roaming/*/raru.exe
```

Analyse the network capture

```
# Full HTTP summary
tshark -r satan_capture.pcap \
  -Y "http" \
  -T fields \
  -e ip.dst \
  -e http.host \
  -e http.request.uri \
  -e http.request.method 2>/dev/null

# Isolate the C2 POST
tshark -r satan_capture.pcap \
  -Y "http.request.method == POST" \
  -T fields \
  -e ip.dst -e http.host -e http.request.uri 2>/dev/null
```

Expected output:

```
149.28.244.165   xhj4hypdsb3jozwn.onion.pw   /g.php   POST
```

Check registry persistence

```
grep -i "run\|raru\|orcy" \
  ~/.wine/system.reg ~/.wine/user.reg 2>/dev/null
```

Act 5 — The Punchline

Static vs Dynamic — what each method found

Indicator	Static Analysis	Dynamic Analysis
File type (PE32)	✓	✓ confirmed
Compile date (2017)	✓	—
Packed / high entropy	✓	—
Anti-debug checks	✓ imports visible	✓ actively evading
Encryption APIs	✓ Advapi present	—

Indicator	Static Analysis	Dynamic Analysis
Drops persistence exe	✗ unknown	✓ Roaming/Orcy/raru.exe
Executes BAT file	✗ unknown	✓ tmp_31ba24ff.bat
Deletes shadow copies	✗ unknown	✓ vssadmin called
C2 communication	✗ unknown	✓ POST /g.php → 149.28.244.165
Randomised naming	✗ unknown	✓ different path every run
Registry persistence	✗ unknown	✓ Run key written

Network IOCs

IOC	Type	Value
C2 domain	DNS	`xhj4hypdsb3jozwn.onion.pw`
C2 IP	IPv4	`149.28.244.165` (AS-VULTR)
C2 endpoint	HTTP POST	`/g.php`
C2 port (initial)	TCP	`80`
C2 port (follow-up)	TCP	`443`

> **Note:** The C2 uses a Tor2Web proxy (`.onion.pw`) so the actual C2 is a Tor hidden service. Satan doesn't require the Tor client on the victim machine — it routes through a public proxy instead.

Host IOCs

IOC	Type	Value
Dropped EXE (Win7)	File path	`AppData\Roaming\Zoumot\ukist.exe`
Dropped EXE (Win10)	File path	`AppData\Roaming\Kased\zihy.exe`
Dropped EXE (REMnux)	File path	`AppData\Roaming\Orcy\raru.exe`

IOC	Type	Value
Dropped EXE SHA256	Hash	`eaa6b8b226fac260a71202de5356898a7e4fc2408625c2c29407aa4b42851449`
BAT file (Win10)	Hash	`d806bafae1c3050f63e8c48ecfdc6e46b7e2de46100d8ec8ca5bbb6ae952b3b7`
Persistence key	Registry	`HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\{A77B256D-...}`

Win7 vs Win10 Behaviour Comparison

Same binary. Same hash. Dramatically different behaviour across OS environments.

Behaviour	Win7	Win10
Injected processes	4	18
Shadow copy deletion	Not confirmed	✓ vssadmin fired
C2 contact	Suricata sig only	✓ Actual HTTP POST confirmed
Registry Run key	Not confirmed	✓ Written
Suricata alerts	1	17
Total processes spawned	47	140

Why the difference? Satan checks the OS version and adjusts behaviour accordingly. Win10 has more processes to inject into, more reliable VSS to delete, and Win10 was the primary target OS when this sample was compiled.

Teaching point: This is why analysts run samples across multiple OS environments. A sample that looks partially contained on Win7 may be fully destructive on Win10.

Wine vs Real Windows — Limitations

Running PE malware under Wine on Linux has known limitations worth flagging:

Behaviour	Real Windows	Wine
Process injection	Full 18-process chain	Partial / fails silently

Behaviour	Real Windows	Wine
Shadow copy deletion	vssadmin executes	Fails (no VSS on Linux)
Registry persistence	Written to real registry	Written to `~/.wine/*.reg`
C2 communication	Full HTTP stack	✓ Works — confirmed live
File drops	To real filesystem	To `~/.wine/drive_c/`

Wine is suitable for initial triage and network IOC collection. For complete behavioural analysis, use a real Windows VM (FlareVM, Windows Defender disabled).

Emergency Commands

```
# Kill malware if it keeps running
kill -9 $(pgrep -f raru.exe)
kill -9 $(pgrep -f wineserver)

# Verify C2 connections are gone
ss -tunp | grep 149.28.244.165

# Bring network back up after demo
sudo ip link set enp0s3 up

# Wipe Wine environment completely
rm -rf ~/.wine

# Reinitialise clean Wine
winecfg
```

Key Takeaways

1. **Static analysis is safe and fast** — run it first, always. It gives you capabilities and indicators without risk.
2. **Dynamic analysis is where the truth is** — behaviour that's hidden from static view (randomised naming, C2 calls, BAT execution) only appears at runtime.
3. **Snapshot before you detonate** — every single time, no exceptions.
4. **Environment matters** — the same binary behaves differently across OS versions. Always test on multiple environments.
5. **Wine is a triage tool, not a lab** — great for quick dynamic triage and network IOC collection, not a replacement for a real Windows analysis VM.

6. **The C2 was live in 2026** — a 2017 ransomware sample still had active infrastructure. Age doesn't mean safe.

Demo performed on REMnux | Satan Ransomware SHA256: 28f299818268701eb239974b74b3d80fc28e2d14cc0f45a52a9bc62be784d170